

Specialist Course: Lattice Field Theory
Final Project: Comparison of Algorithms

Fiona Pärli

December 27, 2025

Contents

1	Introduction	3
2	Scalar ϕ^4 Theory	4
2.1	The ϕ^4 Theory Action	4
2.2	Different Masses m^2	4
2.3	Discretization of Scalar ϕ^4 Theory	5
3	Algorithms	6
3.1	General Introduction to Monte Carlo Methods	6
3.1.1	Example of a Simple Monte Carlo Integral	6
3.1.2	Importance Sampling	7
3.1.3	Markov Chains	8
3.2	The Metropolis Algorithm	8
3.3	The Hybrid Monte Carlo Algorithm (HMC)	9
4	Integrators for HMC	12
4.1	Leapfrog Integration	12
4.2	The Omelyan2 Integrator	13
5	Statistical Analysis	15
5.1	Estimating Errors	15
5.2	Autocorrelation and Integrated Autocorrelation Time	16
5.2.1	The Binning Method	17
5.2.2	Other Methods for Estimating the Autocorrelation	17
6	Topological Charge for the Scalar ϕ^4 Theory in 2D	18
6.1	Kinks and Topology	18
6.2	Periodic Boundary Conditions (PBC)	18
6.3	Aperiodic Boundary Conditions (APBC)	19
7	Results	20
7.1	Details about the Implementation	20
7.1.1	Description of the Python Code	20
7.1.2	Description of the Rust Code	20
7.2	Tuning the Algorithms to the Correct Acceptance Rate	21
7.3	Monte Carlo Chain Visualization	21
7.4	Computational Performance of the Algorithms	22
7.5	Measuring Errors and Autocorrelations	24
7.5.1	Errors	24
7.5.2	Autocorrelations	25
7.6	Results with the Topological Charge	25

Chapter 1

Introduction

This final project of the Specialist Course "Lattice Field Theory" (by Prof. Dr. Fernando Romero-Lopez and Victor Granados) aims to do a comparison of the Metropolis and the HMC algorithms applied to scalar ϕ^4 theory.

write what I
actually did
in the end

Chapter 2

Scalar ϕ^4 Theory

Scalar ϕ^4 theory is the simplest relativistic system in which we can see spontaneous symmetry breaking of discrete symmetries ($\mathbb{Z}_2$ reflection symmetry, $\phi(x) \rightarrow -\phi(x)$). The difference between explicit and spontaneous breaking of a symmetry is that with spontaneous symmetry breaking, the Lagrangian is invariant under the symmetry, but the ground state is not. In explicit symmetry breaking, the symmetry was never there to begin with. In spontaneous symmetry breaking, a phase transition occurs. For discrete symmetries, spontaneous symmetry breaking looks very similar to explicit symmetry breaking. Breaking of a continuous global symmetry (for example a global $U(1)$ phase symmetry) would lead to massless particles (Goldstone theorem) and for gauged symmetries, this could explain the Higgs mechanism. However, this is beyond the scope of this project. This section is provided for completeness and is heavily based on the tutorial of the LFT course and the following book: [11].

2.1 The ϕ^4 Theory Action

We work in two-dimensional Euclidian space. Then, the ϕ^4 - theory action is described by the following Lagrangian:

$$S[\phi] = \int d^2x \left[\frac{1}{2}(\partial_\mu \phi)(\partial^\mu \phi) + \frac{1}{2}m^2\phi^2 + \frac{1}{4!}\lambda\phi^4 \right], \quad (2.1.1)$$

where m^2 and λ are the parameters of the theory. We could regard these parameters as being temperature dependent, then this model could describe a system such as a ferromagnet near its critical temperature.

We assume periodic boundary conditions in all dimensions, hence by integrating by parts, we obtain

$$S[\phi] = \int d^2x \left[-\frac{1}{2}\phi(\partial_\mu\partial^\mu\phi) + \frac{1}{2}m^2\phi^2 + \frac{1}{4!}\lambda\phi^4 \right]. \quad (2.1.2)$$

The total derivative term vanished because of the periodic boundary conditions. This rewriting shows explicitly that the kinetic energy term becomes a nearest neighbour coupling when discretizing the action.

2.2 Different Masses m^2

The action has the following scalar potential:

$$V(\phi) = \frac{1}{2}m^2\phi^2 + \frac{1}{4!}\lambda\phi^4. \quad (2.2.1)$$

The parameter λ has the same dimension as a squared mass. We assume that $\lambda > 0$, such that the potential is bounded from below. This is required to define a proper distribution, as it ensures that the Boltzmann weight is normalizable. This potential can model a system near a phase transition, such as a ferromagnet near the critical temperature T_c . The mass parameter m^2 plays a role analogous to the inverse correlation length, its sign determines the phase of the system. We can imagine the mass to be temperature dependent:

$$m^2(T) = c(T - T_c), \quad (2.2.2)$$

where c is some constant [11]. We distinguish for the following two cases for the mass parameter:

- If $m^2 \geq 0$ ($T \geq T_c$), the potential has a minimum at $\phi = 0$, which corresponds to a minimum of the action. There is a single vacuum state for which $\langle \phi \rangle = 0$.
- If $m^2 < 0$ ($T < T_c$), $\phi = 0$ is a local maximum and there are two global minima at $\phi = \pm \sqrt{-\frac{6m^2}{\lambda}}$. There are now two vacuum states and $\langle \phi \rangle = \pm \sqrt{-\frac{6m^2}{\lambda}}$.

Since the vacuum expectation value $\langle \phi \rangle = 0$ is invariant under $\phi \rightarrow -\phi$, but the two vacua $\langle \phi \rangle = \pm \sqrt{-6m^2/\lambda}$ are not, this is a case of spontaneous symmetry breaking. That means there occurs a phase transition at some critical mass value $m_c^2 < 0$. At this critical point, the correlation length diverges. This is the point where Monte-Carlo simulations become critical-slowing-down limited.

In this picture, spontaneous symmetry breaking is associated with a continuous phase transition where the effective mass crosses zero. The order parameter $\langle \phi \rangle$ changes from zero in the symmetric phase to a nonzero value in the broken phase, analogous to the magnetization of a ferromagnet emerging below the Curie temperature.

In Euclidian lattice simulations, the temperature does not appear explicitly (where for example in the Ising model, it does). Changing λ and m^2 plays the role of changing external thermodynamical variables. This also shows that the scalar ϕ^4 theory with its spontaneous $\mathbb{Z}_2$ symmetry breaking mirrors the magnetization transition of the Ising model. For more details about the Ising model, see [14].

2.3 Discretization of Scalar ϕ^4 Theory

This section is based on the tutorial of the LFT course.

We discretize this theory on a two-dimensional spacetime lattice of L^2 lattice sites with lattice spacing a and denote the spacetime points as

$$\vec{x} = a\vec{n} = a(n_1, n_2), \quad 0 \leq n_1, n_2 < L. \quad (2.3.1)$$

We approximate the integral as a sum over all lattice sites:

$$\int d^2x \rightarrow a^2 \sum_{\vec{n}}, \quad (2.3.2)$$

For the discretization of the second derivative, we use the five-point stencil discretization (more on that, see [15]):

$$\begin{aligned} \partial_\mu \partial^\mu \phi(\vec{x}) &= \left(\frac{\partial^2}{\partial x_1^2} + \frac{\partial^2}{\partial x_2^2} \right) \phi(\vec{x}) \\ &\rightarrow \frac{\phi(n_1+1, n_2) + \phi(n_1-1, n_2) + \phi(n_1, n_2+1) + \phi(n_1, n_2-1) - 4\phi(\vec{n})}{a^2} + \mathcal{O}(a^2) \\ &= \sum_{\mu \in \{1,2\}} \frac{1}{a^2} [\phi(\vec{n} + \hat{\mu}) + \phi(\vec{n} - \hat{\mu}) - 2\phi(\vec{n})] + \mathcal{O}(a^2). \end{aligned} \quad (2.3.3)$$

All together, the discretized action reads

$$S[\phi] = a^2 \sum_{\vec{n}} \left(-\frac{1}{2a^2} \phi(\vec{n}) \sum_{\mu} [\phi(\vec{n} + \hat{\mu}) + \phi(\vec{n} - \hat{\mu}) - 2\phi(\vec{n})] + \frac{1}{2} m^2 \phi^2(\vec{n}) + \frac{1}{4!} \lambda \phi^4(\vec{n}) \right) \quad (2.3.4)$$

or, after some algebra,

$$S[\phi] = \sum_{\vec{n}} \left(\left[2 + \frac{1}{2} m^2 \right] \phi^2(\vec{n}) + \frac{1}{4!} \lambda \phi^4(\vec{n}) - \sum_{\mu} [\phi(\vec{n} + \hat{\mu}) \phi(\vec{n})] \right) \quad (2.3.5)$$

where we have redefined $(am)^2 \rightarrow m^2$ and $(a^2\lambda) \rightarrow \lambda$. The discretized action has a changed mass, $m_{\text{eff}}^2 = 2 + \frac{1}{2} m^2$.

Chapter 3

Algorithms

3.1 General Introduction to Monte Carlo Methods

Our goal is to evaluate the expectation value of an observable $O[U]$ in the quantized Euclidian gauge field theory on a lattice. This is written as a path integral (functional)

$$\langle O \rangle = \frac{1}{Z} \int \mathcal{D}[U] \exp(-S[U]) O[U], \quad (3.1.1)$$

where the partition function is defined as

$$Z = \int \mathcal{D}[U] \exp(-S[U]). \quad (3.1.2)$$

In the general case, these integrals cannot be solved analytically and depending on the application, these can be high-dimensional [8].

Using probability theory, we can approximate the integral of a function $f(x)$ by averaging the functional values $f(x_n)$ at values x_n , which are randomly chosen according to a uniform distribution

$$\rho_u(x_n) = \frac{1}{b-a}. \quad (3.1.3)$$

The integral is approximated as:

$$\frac{1}{b-a} \int_a^b dx f(x) = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{n=1}^N f(x_n). \quad (3.1.4)$$

The exact mean is replaced by the sample mean, the calculation is restricted to a subsample and a finite number N . By the Strong Law of Large Numbers, the Monte Carlo estimator

$$\frac{1}{N} \sum_{n=1}^N f(x_n)$$

converges almost surely to the exact expectation value as $N \rightarrow \infty$, and the Central Limit Theorem implies an error scaling of $\mathcal{O}\left(\frac{1}{\sqrt{N}}\right)$. This means to increase the accuracy by a factor of two, one has to use four times as many samples [8].

3.1.1 Example of a Simple Monte Carlo Integral

Suppose we want to estimate the integral

$$\int_0^1 \exp(-x^2) dx. \quad (3.1.5)$$

We restrict the upper limit to 1 to match the support of a uniform random variable. Let X be a random variable uniformly distributed on the interval $(0, 1)$. Its probability density function (pdf) is given by

$$f(x) = \begin{cases} 1 & \text{if } x \in (0, 1), \\ 0 & \text{otherwise.} \end{cases} \quad (3.1.6)$$

Then the integral can be written as an expectation,

$$\int_0^1 \exp(-x^2) dx = \mathbb{E}[\exp(-X^2)]. \quad (3.1.7)$$

A Monte Carlo estimator using N independent samples $X_1, X_2, \dots, X_N$ from the uniform distribution is

$$I_N = \frac{1}{N} \sum_{i=1}^N \exp(-X_i^2). \quad (3.1.8)$$

This estimator converges to the true value as $N \rightarrow \infty$,

$$I_N \xrightarrow{N \rightarrow \infty} \int_0^1 \exp(-x^2) dx.$$

This simple example illustrates how Monte Carlo methods approximate integrals by replacing them with sample averages [1].

3.1.2 Importance Sampling

In our path integral in Equation 3.1.1, we need to account for the Boltzmann factor $\exp(-S)$. This is a weighting factor, depending on the action S , of the different field configurations. The space of all configurations is not uniformly explored, but according to the weights determined by the action S . The main idea of importance sampling is to approximate a large sum with a comparatively small amount of configurations, sampled according to this weight factor.

For a general probability distribution $\rho(x)$, the expectation value of a function $f(x)$ is

$$\langle f \rangle_\rho = \frac{\int_a^b dx \rho(x) f(x)}{\int_a^b dx \rho(x)}. \quad (3.1.9)$$

In the importance sampling Monte Carlo integration, this is approximated as follows:

$$\langle f \rangle_\rho = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{n=1}^N f(x_n), \quad (3.1.10)$$

where $x \in (a, b)$ sampled according to the normalized probability density $\frac{\rho(x)dx}{\int_a^b dx \rho(x)}$. Our path integral (Equation 3.1.1) is of this form, so we can apply this method:

$$\langle O \rangle = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{n=1}^N O[U_n], \quad (3.1.11)$$

with U_n sampled according to the probability distribution density

$$dP(U) = \frac{e^{-S[U]} \mathcal{D}[U]}{\int \mathcal{D}[U] e^{-S[U]}}, \quad (3.1.12)$$

which is called the Gibbs-measure [8].

3.1.3 Markov Chains

The configurations U_n are generated in a sequence, a so-called homogeneous Markov chain: $U_0 \rightarrow U_1 \rightarrow U_2 \rightarrow \dots$. The first configuration U_0 is chosen randomly, the update from one configurations to the next is called the Markov step. The next configuration only depends on the present one, not on the history of the process. This condition is written as

$$P(U_n = U' | U_{n-1} = U) = T(U' | U). \quad (3.1.13)$$

The configuration sequence explores the space of all configurations according to the probability density defined by the action. Eventually, the Markov chain will reach an equilibrium, after that, it cannot have sinks or sources of probability. This gives rise to a balancing condition:

$$\sum_U T(U | U') P(U) = \sum_U T(U' | U) P(U'). \quad (3.1.14)$$

A solution for the balancing condition in Equation 3.1.14 can be obtained by requiring term-wise equality:

$$T(U | U') P(U) = T(U' | U) P(U'). \quad (3.1.15)$$

This is called detailed balance condition, it is the most commonly used solution. Using the normalization property, we find in addition that

$$P(U') = \sum_U T(U | U') P(U). \quad (3.1.16)$$

The equilibrium distribution $P(U)$ is a fixed point of the Markov process, once the equilibrium is reached, the system stays there forever.

In order to give correct result, the Markov chain must be aperiodic, i.e. be able to reach all configurations in the configuration space. This is the case iff the transition matrix $T(U | U')$ is positive-semidefinite. This property is called strong ergodicity [8].

3.2 The Metropolis Algorithm

The Metropolis Algorithm is used to obtain a new configuration U_{n+1} from the current configuration U_n . We use $P[U] \propto \exp(S[U])$. The steps are as follows:

1. Choose a candidate configuration U' according to some a priori selection probability $T_0(U' | U)$, where $U = U_n$.
2. Accept the candidate configuration U' as the new configuration U_{n+1} with the following probability:

$$T_A(U' | U) = \min \left(1, \frac{T_0(U | U') \exp(-S[U'])}{T_0(U' | U) \exp(-S[U])} \right). \quad (3.2.1)$$

If the change is rejected, keep the old configuration, set $U_{n+1} = U_n$.

3. Repeat.

The detailed balance condition is fulfilled:

$$T(U' | U) \exp(-S[U]) = T_0(U' | U) \min \left(1, \frac{T_0(U | U') \exp(-S[U'])}{T_0(U' | U) \exp(-S[U])} \right) \exp(-S[U]) \quad (3.2.2)$$

$$= \min (T_0(U | U') \exp(-S[U']), T_0(U' | U) \exp(-S[U])) \quad (3.2.3)$$

$$= T(U | U') \exp(-S[U']) \quad (3.2.4)$$

We have used the symmetry of min and the positivity of all factors.

Often, we have $T_0(U | U') = T_0(U' | U)$. In this case, the transition probability simplifies to

$$T_A(U' | U) = \min (1, \exp(-\Delta S)), \quad \text{with} \quad \Delta S = S[U'] - S[U]. \quad (3.2.5)$$

The decision to accept or reject only depends on the change in the action between the two configurations. Any normalization constant in $P[U]$ will cancel automatically, note that this is crucial in field theory where the partition function Z is unknown. In practice, the acceptance rate is strongly dependent on the proposed change. Very large proposed updates are often rejected, very small ones lead to long autocorrelation times. Hence we often aim for acceptance rates between 50% - 80%.

A warm-up period (thermalization) is required to eliminate dependence on the initial configuration. After that, configurations are still correlated, and one must estimate autocorrelation times when computing errors.

It can be shown that the Metropolis algorithm fulfills ergodicity, meaning that any configuration is reachable from any other configuration with positive probability in a finite number of steps. Because the transition kernel satisfies detailed balance with respect to $\exp(-S[U])$, this probability distribution is the stationary distribution of the Markov chain. Together with ergodicity, this ensures convergence of the algorithm to the desired ensemble.

3.3 The Hybrid Monte Carlo Algorithm (HMC)

The Hybrid Monte Carlo Algorithm (HMC) algorithm was first introduced in this paper: [6]. The authors solved the major computational challenge of involving fermions into lattice field theory.

Simulating fermions in a lattice field theory is difficult, since fermions are described by Grassmann numbers, for which Monte Carlo methods cannot be applied directly. Integrating over fermions produces very non-local effects in the action, which makes calculations extremely expensive. To solve this problem, the nested Monte Carlo steps are replaced with an equation of motion in a fictitious time variable. These are solved numerically with different techniques. Evolving the entire field configuration simultaneously using a numerical integrator avoids the locality problem but introduces truncation errors from discrete integration. Although these errors can be extrapolated away, they remain undesirable. The paper [6] introduces the HMC, which addresses the problem with the following steps:

- evolve all fields in parallel using a molecular-dynamics-like update,
- a global Metropolis accept/reject step, which removes the truncation errors.

This method combines deterministic molecular dynamics and accept/reject steps, achieving efficiency and exactness. Even though this algorithm was originally developed for lattice field theories, it is nowadays applied in various other fields.

The goal is again to calculate the expectation of some operator along with its partition function, as defined in the Equations 3.1.2 and 3.1.2. The Metropolis algorithm is a rather inefficient way to do this and we search for a method to choose candidate configurations efficiently. We aim for a high acceptance rate and minimum correlation between successive configurations. For this, we introduce an artificial computer time τ and a hamiltonian dynamics to describe the development of the field $\phi(\tau)$. In addition, we will need the conjugate momenta $\pi(\tau)$. This means, instead of sampling the distribution $p(\phi) = \frac{1}{Z}e^{-S(\phi)}$, we sample $p(\phi, \pi) = \frac{1}{Z'}e^{-H[\phi, \pi]}$. The Hamiltonian $H[\phi, \pi]$ is given as follows:

$$H[\pi, \phi] = \frac{1}{2}\pi^2 + S(\phi), \quad (3.3.1)$$

where $S(\phi)$ is an arbitrary action. A key point of this algorithm is that the expectation values over $p(\phi)$ are the same as those over $p(\phi, \pi)$ up to some constant, since the integration over $\mathcal{D}\pi$ is a Gaussian integral, which gives rise to a constant. This constant will cancel out in the accept/reject step.

Introducing the conjugate momenta translates the numerical evaluation of the path integral into the language of classical Hamiltonian mechanics. The HMC algorithm solves the classical equations of motion for

each lattice site x :

$$\dot{\phi}_x = \frac{d\phi_x}{d\tau} = \frac{\partial H}{\partial \pi} = \pi_x \quad (3.3.2)$$

$$\dot{\pi}_x = \frac{d\pi_x}{d\tau} = -\frac{\partial H}{\partial \phi_x} = -\frac{\partial S}{\partial \phi_x} = -F[\phi_x] \quad \text{"force"} \quad (3.3.3)$$

Because of the conservation of energy, the Hamiltonian remains constant. This can be used to check the accuracy of the solutions of the differential equations, which have to be solved numerically. In Chapter 4, there will be a description of different methods to achieve this numerical solution.

The procedure for generating a new configuration ϕ' is to select some initial momenta π from a standard Gaussian distribution (zero mean, unit variance)

$$P_G(\pi) \propto \exp\left(-\frac{\pi^2}{2}\right), \quad (3.3.4)$$

and then evolve the system through the (ϕ, π) - phase space for a time τ_0 according to Hamilton's equations 3.3.2. The probability of choosing the configuration (ϕ', π') is

$$P_H((\phi, \pi) \rightarrow (\phi', \pi')) = \delta[(\phi', \pi') - (\phi(\tau_0), \pi(\tau_0))]. \quad (3.3.5)$$

The acceptance probability for this candidate is

$$P_A((\phi, \pi) \rightarrow (\phi', \pi')) = \min(1, \exp(\delta H)), \quad (3.3.6)$$

where $\delta H = H(\phi', \pi') - H(\phi, \pi)$.

In order for the HMC to correctly sample the target distribution $p(\phi) \propto e^{-S(\phi)}$, the transition probability restricted to the field variables ϕ alone must satisfy detailed balance. Starting from the full transition probability in (ϕ, π) space, the transition probability for $\phi \rightarrow \phi'$ is obtained by integrating out the momenta:

$$P_M(\phi \rightarrow \phi') = \int [d\pi] [d\pi'] P_G(\pi) P_H((\phi, \pi) \rightarrow (\phi', \pi')) P_A((\phi, \pi) \rightarrow (\phi', \pi')). \quad (3.3.7)$$

Here $P_G(\pi)$ is the Gaussian distribution used to refresh the momenta, P_H is the deterministic Hamiltonian evolution, and P_A is the Metropolis accept/reject step.

The main requirement for detailed balance is that the molecular-dynamics evolution is reversible. This means that evolving (ϕ, π) forward in fictitious time τ to (ϕ', π') is equivalent to evolving $(\phi', -\pi')$ backward to $(\phi, -\pi)$:

$$P_H((\phi, \pi) \rightarrow (\phi', \pi')) = P_H((\phi', -\pi') \rightarrow (\phi, -\pi)). \quad (3.3.8)$$

This property holds automatically for Hamiltonian dynamics because the flow generated by a Hamiltonian is time-reversal invariant when accompanied by a sign flip of all momenta. From the following identity (which we already used to show detailed balance for Metropolis)

$$e^{-H(\phi, \pi)} \min(1, e^{-\Delta H}) = \min(e^{-H(\phi, \pi)}, e^{-H(\phi', \pi')}) \quad (3.3.9)$$

$$= e^{-H(\phi', \pi')} \min(1, e^{+\Delta H}), \quad (3.3.10)$$

the fact that the Hamiltonian $H(\phi, \pi)$ is also invariant under $\pi \rightarrow -\pi$ and $P_S(\phi)P_G(\pi) \propto \exp(H(\phi, \pi))$, we obtain

$$P_S(\phi) P_G(\pi) P_A((\phi, \pi) \rightarrow (\phi', \pi')) = P_S(\phi') P_G(\pi') P_A((\phi', \pi') \rightarrow (\phi, \pi)). \quad (3.3.11)$$

Multiplying this by the transition probability P_H and integrating over π, π' gives

$$\int [d\pi] [d\pi'] P_S(\phi) P_G(\pi) P_H((\phi, \pi) \rightarrow (\phi', \pi')) P_A((\phi, \pi) \rightarrow (\phi', \pi')) \quad (3.3.12)$$

$$= \int [d\pi] [d\pi'] P_S(\phi') P_G(\pi') P_H((\phi', \pi') \rightarrow (\phi, \pi)) P_A((\phi', \pi') \rightarrow (\phi, \pi)). \quad (3.3.13)$$

Because the measure is invariant under momentum reversal,

$$[d\pi][d\pi'] = [d(-\pi)][d(-\pi')],$$

and using the reversibility condition from above, this reduces precisely to

$$P_S(\phi) P_M(\phi \rightarrow \phi') = P_S(\phi') P_M(\phi' \rightarrow \phi), \quad (3.3.14)$$

which is the detailed balance condition for P_M .

By conservation of energy, we need $H = H'$, meaning $\delta H = 0$ and $P_A = 1$. However, solving differential equations numerically always introduces errors, hence in general, $P_A \leq 1$. We have just proved that the HMC produces ϕ configurations with the correct distribution for any $\delta H \neq 0$. Also note that the momenta π are not used to compute the observables. They can be disregarded in each step [6].

Note that the accept/reject step plays a crucial role in the HMC. Hamiltonian evolution exchanges energy between the kinetic term $\frac{1}{2}\pi^2$ and the potential term $S(\phi)$ while (ideally) keeping their sum $H = \frac{1}{2}\pi^2 + S(\phi)$ constant. The accept/reject step corrects the small energy violations $\delta H \neq 0$ resulting from the numerical integration, as it compares the Hamiltonians and accepts the proposed configuration with probability P_A . This enforces the correct equilibrium distribution, as a substantial increase in the energy will be rejected. In addition, the accept/reject step also ensures ergodicity. Pure Hamiltonian evolution is deterministic and reversible, and without momentum refreshment or a mechanism to leave a given energy surface, the algorithm would explore only a single trajectory in phase space. Periodically resampling the momenta from a Gaussian distribution redistributes kinetic energy, allowing the system to have different energies [6].

Chapter 4

Integrators for HMC

Our aim is to solve the Hamiltonian equations of motion 3.3.2, which are at the heart of the HMC algorithm, numerically. This Chapter describes different methods used for this task.

4.1 Leapfrog Integration

In Section 3.3, we have discussed that the detailed balance condition for the HMC algorithm imposes the following two requirements on the molecular dynamics trajectory:

- Invariance of the integration measure under Hamiltonian evolution in the (ϕ, π) phase space:

$$[\mathcal{D}\phi][\mathcal{D}\pi] = [\mathcal{D}\phi(\tau)][\mathcal{D}\pi(\tau)] \quad (4.1.1)$$

- Reversibility of the trajectory, evolving forward in fictitious time τ and then reversing the momenta exactly retraces the trajectory

The leapfrog integration method fulfills these requirements, as will be shown in the following. Leapfrog is a second-order method, compared to the very-well known Euler method.

In the leapfrog integration method, the field ϕ is evolved in n steps of length ε . The conjugate momenta start with a half-step of $\frac{\varepsilon}{2}$, then $(n - 1)$ full steps and finally other half-step. A single such step ($n = 1$) is denoted as follows:

$$\phi(0) \rightarrow \phi(\varepsilon) \quad (4.1.2)$$

$$\pi(0) \rightarrow \pi(\frac{\varepsilon}{2}) \rightarrow \pi(\varepsilon). \quad (4.1.3)$$

The simplest integration scheme is second-order accurate:

$$\phi(0) \rightarrow \phi(\varepsilon) = \phi(0) + \pi(\frac{\varepsilon}{2})\varepsilon \quad (4.1.4)$$

$$\pi(0) \rightarrow \pi(\frac{\varepsilon}{2}) = \pi(0) - \frac{\partial S}{\partial \phi}|_{\phi(0)}\frac{\varepsilon}{2} \rightarrow \pi(\varepsilon) = \pi(\frac{\varepsilon}{2}) - \frac{\partial S}{\partial \phi}|_{\phi(\varepsilon)}\frac{\varepsilon}{2} \quad (4.1.5)$$

When dealing with a series of steps of length n , we use the abbreviations $\phi(n\varepsilon) = \phi_n$ and $\pi(n\varepsilon) = \pi_n$.

The invariance of the integration measure can be shown from the product of Jacobians along the sequence:

$$\det \left[\frac{\partial(\pi_1, \phi_1)}{\partial(\pi_0, \phi_0)} \right] = \det \left[\frac{\partial(\pi_1, \phi_1)}{\partial(\pi_{\frac{1}{2}}, \phi_1)} \frac{\partial(\pi_{\frac{1}{2}}, \phi_1)}{\partial(\pi_{\frac{1}{2}}, \phi_0)} \frac{\partial(\pi_{\frac{1}{2}}, \phi_0)}{\partial(\pi_0, \phi_0)} \right] = 1 \quad (4.1.6)$$

This factorizes into a product of three determinants of upper diagonal matrices with 1 on the diagonal, hence the result equals 1 and the integration measure is invariant.

For the reversibility property, use the scheme in Equations 4.1.4 starting from (ϕ_0, π_0) :

$$\phi_1 = \phi_0 + \pi_0 \varepsilon - \frac{1}{2} \frac{\partial S}{\partial \phi} \Big|_{\phi_0} \varepsilon^2 \quad (4.1.7)$$

$$\pi_1 = \pi_0 - \frac{1}{2} \left(\frac{\partial S}{\partial \phi} \Big|_{\phi_0} + \frac{\partial S}{\partial \phi} \Big|_{\phi_1} \right) \varepsilon \quad (4.1.8)$$

To go backwards, we start at (ϕ_1, π_1) and apply the equations of motion 4.1.4 with a negative step size $-\varepsilon$. This gives

$$\phi(\varepsilon - \varepsilon) = \phi_1 - \pi_1 \varepsilon - \frac{1}{2} \frac{\partial S}{\partial \phi} \Big|_{\phi_1} \varepsilon^2 \quad (4.1.9)$$

$$= \phi_1 - \pi_1 \varepsilon + \frac{1}{2} \left(\frac{\partial S}{\partial \phi} \Big|_{\phi_0} + \frac{\partial S}{\partial \phi} \Big|_{\phi_1} \right) \varepsilon^2 - \frac{1}{2} \frac{\partial S}{\partial \phi} \Big|_{\phi_1} \varepsilon^2 \quad (4.1.10)$$

$$= \phi_1 - \pi_1 \varepsilon + \frac{1}{2} \frac{\partial S}{\partial \phi} \Big|_{\phi_0} \varepsilon^2 = \phi_0, \quad (4.1.11)$$

$$\pi(\varepsilon - \varepsilon) = \pi_0 = \pi_1 + \left(\frac{\partial S}{\partial \phi} \Big|_{\phi_1} + \frac{1}{2} \frac{\partial S}{\partial \phi} \Big|_{\phi_0} \right) \varepsilon = \pi_0. \quad (4.1.12)$$

So the reversibility is proven. It is possible to combine n leapfrog steps. The discretization error for half-steps is $\mathcal{O}(\varepsilon^2)$, for full steps $\mathcal{O}(\varepsilon^3)$ [8].

4.2 The Omelyan2 Integrator

This integrator is described in great detail in the following papers: [12, 9].

The Omelyan2 integrator is a generalization of the standard leapfrog scheme, designed to improve energy conservation while preserving the two key properties required for HMC:

- Invariance of the integration measure in the (ϕ, π) phase space
- Reversibility of the trajectory under time reversal

Like leapfrog, Omelyan integration evolves the fields ϕ and conjugate momenta π in a sequence of small steps. The key difference is that the momentum and position updates are distributed asymmetrically within each step according to a free parameter λ , which can be tuned to minimize the discretization error of the Hamiltonian.

A single Omelyan step of size ε can be written as:

$$\pi \rightarrow \pi - \lambda \frac{\varepsilon}{2} \frac{\partial S}{\partial \phi} \quad (4.2.1)$$

$$\phi \rightarrow \phi + \frac{\varepsilon}{2} \pi \quad (4.2.2)$$

$$\pi \rightarrow \pi - (1 - 2\lambda) \frac{\varepsilon}{2} \frac{\partial S}{\partial \phi} \quad (4.2.3)$$

$$\phi \rightarrow \phi + \frac{\varepsilon}{2} \pi \quad (4.2.4)$$

$$\pi \rightarrow \pi - \lambda \frac{\varepsilon}{2} \frac{\partial S}{\partial \phi} \quad (4.2.5)$$

Here, λ is a free parameter, and the standard leapfrog scheme is recovered for $\lambda = 0$. In practice, a common choice is $\lambda \approx 0.1931833$, which minimizes the error in the Hamiltonian for harmonic systems [12, 9].

The Omelyan 2 integrator improves on the basic leapfrog scheme in the following ways:

- Reduced energy drift: By distributing the momentum updates asymmetrically, the integrator achieves better conservation of the Hamiltonian over long trajectories, which increases the acceptance rate in HMC.

- Preservation of measure and reversibility: Like leapfrog, the Omelyan 2 integrator is reversible, ensuring that detailed balance in HMC is maintained.
- Higher efficiency: For a given step size ε , Omelyan2 typically allows longer trajectories or fewer steps while achieving the same or better acceptance, reducing computational cost.

Chapter 5

Statistical Analysis

In LFT, every calculated observable needs a statistical analysis to estimate its uncertainty, quantities without error estimations are meaningless.

5.1 Estimating Errors

The variance of a random variable X is defined as

$$\text{Var}(X) = E[(X - E[X])^2]. \quad (5.1.1)$$

In our case, we use as the random variable the expectation value of some observable, which we aim to calculate, $X = \langle \mathcal{O} \rangle$. Hence we aim to calculate

$$E[(\langle \mathcal{O} \rangle - E[\mathcal{O}])^2] = E \left[\left(\frac{1}{N} \sum_{i=1}^N (\mathcal{O}_i - E[\mathcal{O}]) \right)^2 \right], \quad (5.1.2)$$

where we expand the square as follows:

$$\frac{1}{N^2} \sum_{i,j=1}^N E[(\mathcal{O}_i - E[\mathcal{O}](\mathcal{O}_j - E[\mathcal{O}]))]. \quad (5.1.3)$$

The expectation value in the sum is precisely the covariance

$$C_{ij}[\mathcal{O}] := E[\mathcal{O}_i \mathcal{O}_j] - E[\mathcal{O}]^2. \quad (5.1.4)$$

So the variance of the sample mean can be written as

$$E[(\langle \mathcal{O} \rangle - E[\mathcal{O}])^2] = \frac{1}{N^2} \sum_{i,j=1}^N C_{ij}[\mathcal{O}]. \quad (5.1.5)$$

The next step is to separate the diagonal from the off-diagonal terms. The diagonal terms $i = j$ give

$$\sum_{i=1}^N C_{ii}[\mathcal{O}] = N \text{Var}[\mathcal{O}]. \quad (5.1.6)$$

The off-diagonal terms encode the correlations between the different measurements. Thus, in total we obtain

$$\frac{1}{N^2} \sum_{i,j=1}^N C_{ij}[\mathcal{O}] = \frac{\text{Var}[\mathcal{O}]}{N} + \frac{1}{N^2} \sum_{i \neq j} C_{ij}[\mathcal{O}]. \quad (5.1.7)$$

For a stationary Markov process, the correlations only depend on the time separation

$$t = |i - j|, \quad C_{ij}[\mathcal{O}] = C_{1,1+t}[\mathcal{O}]. \quad (5.1.8)$$

For a fixed t , there are $N - t$ pairs (i, j) with $j = i + t$. Since $C_{ij} = C_{ji}$, we have a factor of 2, giving

$$\sum_{i \neq j} C_{ij}[\mathcal{O}] = 2 \sum_{t=1}^{N-1} (N - t) C_{1,t+1}[\mathcal{O}]. \quad (5.1.9)$$

Putting all the parts together, we obtain the following formula:

$$\mathbb{E} \left[(\langle \mathcal{O} \rangle - \mathbb{E}[\mathcal{O}])^2 \right] = \frac{\text{Var}[\mathcal{O}]}{N} + \frac{2}{N^2} \sum_{t=1}^{N-1} (N - t) C_{1,t+1}[\mathcal{O}]. \quad (5.1.10)$$

The first term encodes the result for independent samples, where the error is proportional to $\frac{1}{\sqrt{N}}$. The second term encodes the autocorrelation contribution, which increases the statistical error.

Unfortunately, we can only estimate the statistical error through the expression

$$\sigma := \frac{\sigma_{\mathcal{O}}}{\sqrt{N-1}} = \sqrt{\frac{\langle \mathcal{O}^2 \rangle - \langle \mathcal{O} \rangle^2}{N-1}}. \quad (5.1.11)$$

The validity of this estimator comes from the fact that its expectation value matches the statistical error:

$$\begin{aligned} \mathbb{E} [\langle \mathcal{O}^2 \rangle - \langle \mathcal{O} \rangle^2] &= (N-1) \frac{\text{Var}[\mathcal{O}]}{N} - \frac{2}{N^2} \sum_{t=1}^{N-1} (N-t) C_{1,t+1}[\mathcal{O}] \\ &\stackrel{\text{ind.}}{=} (N-1) \frac{\text{Var}[\mathcal{O}]}{N} \\ &\implies \mathbb{E} \left[\frac{\langle \mathcal{O}^2 \rangle - \langle \mathcal{O} \rangle^2}{N-1} \right] = \frac{\text{Var}[\mathcal{O}]}{N}. \end{aligned} \quad (5.1.12)$$

[3, 4, 7]

5.2 Autocorrelation and Integrated Autocorrelation Time

In typical Monte Carlo methods in LFT, the configurations are always correlated, since we only make small steps while exploring the space of possible states. In our case, we have $\phi_{i+1} \in [\phi_i - w, \phi_i + w]$, where w is the width. Elements later in the chain, for example ϕ_{i+100} should be more independent from ϕ_i . We will try to find the number of samples that can be considered independent, which is the idea behind autocorrelation. We rewrite the formula from above as follows:

$$\begin{aligned} \mathbb{E} \left[(\langle \mathcal{O} \rangle - \mathbb{E}[\mathcal{O}])^2 \right] &= \frac{\text{Var}[\mathcal{O}]}{N} + \frac{2}{N^2} \sum_{t=1}^{N-1} (N-t) C_{1,t+1}[\mathcal{O}] \\ &= 2 \left(\frac{1}{2} + \sum_{t=1}^{N-1} \left(1 - \frac{t}{N} \right) \frac{C_{1,t+1}[\mathcal{O}]}{\text{Var}[\mathcal{O}]} \right) \frac{\text{Var}[\mathcal{O}]}{N} \\ &\approx 2\tau_{\text{int}} \frac{\text{Var}[\mathcal{O}]}{N}, \end{aligned} \quad (5.2.1)$$

where

$$\tau_{\text{int}} = \frac{1}{2} + \sum_{t=1}^{N-1} \frac{C_{1,t+1}[\mathcal{O}]}{\text{Var}[\mathcal{O}]} \quad (5.2.2)$$

Here, τ_{int} denotes the integrated autocorrelation time, which is a measure of how many steps there must be between samples to consider them as independent. This depends on the observable and needs to be recalculated for all observables. [2]

5.2.1 The Binning Method

A simple method for estimating the autocorrelation is the binning method, which leaves the mean unchanged but modifies the error estimate to account for the dependence of the configurations.

1. Start with a thermalized chain of N samples of some observable $\mathcal{O}_1, \dots, \mathcal{O}_N$.
2. For a sequence of bin sizes $b_1, \dots, b_N$, do:
 - (a) Divide the chain in non-overlapping bins of size b_i .
 - (b) Compute the average within each bin, producing a chain $\mathcal{O}'_1, \dots, \mathcal{O}'_N$.
 - (c) Treat the binned values as independent and compute the naive error σ .
3. Repeating for all bin sizes gives errors $\sigma_1, \dots, \sigma_N$.
4. Plot the errors as a function of the bin size. For sufficiently large chains, the errors typically plateau as the bin size exceeds the autocorrelation length.
5. The limiting value of this plateau provides a reliable estimate of the statistical error.

[8]

5.2.2 Other Methods for Estimating the Autocorrelation

There are other methods to estimate the autocorrelation, which are more accurate than the binning method. For example:

- Direct calculation of the autocorrelation function
- Jackknife method
- Bootstrapping
- Spectral Method (FFT)
- Overlapping batches mean

[8]

Chapter 6

Topological Charge for the Scalar ϕ^4 Theory in 2D

This chapter summarizes some experiments about the topological charge in a scalar ϕ^4 theory in two dimensions. All of this chapter is based on this paper: [5].

Consider the discretized action of the scalar ϕ^4 theory in Equation 2.3.5.

6.1 Kinks and Topology

In the broken phase of the two-dimensional scalar ϕ^4 theory, the potential possesses two degenerate minima at $\phi = \pm v$. As a consequence, the theory admits non-trivial classical field configurations known as kinks. A kink is a static solution of the field equations that interpolates between the two vacua, satisfying

$$\phi(x \rightarrow -\infty) = -v, \quad \phi(x \rightarrow +\infty) = +v,$$

while an antikink interpolates in the opposite direction. These configurations are distinguished from the vacuum by their topology. In particular, kink configurations cannot be continuously deformed into a constant vacuum configuration without forcing the field to leave the vacuum manifold, which would require infinite energy in the continuum limit. As a result, the space of field configurations decomposes into topologically distinct sectors. The topological properties of the theory are characterized by a conserved topological charge, which is proportional to the difference of the field values at spatial infinity. Vacuum configurations correspond to a trivial topological sector with vanishing charge, while kinks and antikinks carry non-zero topological charge. The existence of these non-trivial sectors depends crucially on the choice of boundary conditions, which determine whether configurations interpolating between different vacua are allowed.

6.2 Periodic Boundary Conditions (PBC)

With periodic boundary conditions (PBC), the fields satisfy

$$\phi(x + L\hat{\mu}) = \phi(x), \tag{6.2.1}$$

for all directions $\hat{\mu}$. In this case, the lattice is topologically trivial, configurations that interpolate between distinct classical vacua are not allowed, because periodicity enforces $\phi(0) = \phi(L)$.

In the broken phase of the ϕ^4 theory, the potential has two degenerate minima $\phi = \pm \sqrt{\frac{-6m^2}{\lambda}} := v$. Even though in infinite volume, spontaneous symmetry breaking occurs, on a finite lattice with PBC, the Monte Carlo average is $\langle \phi \rangle = 0$, because of tunneling between the vacua. As a result $\langle \phi \rangle$ is not a suitable order parameter to distinguish between the symmetric and broken phase.

Nevertheless, simulations with PBC are essential for determining renormalized quantities of the theory. For

instance, the mass m and the field renormalization constant Z can be extracted from the two-point correlation function, while the renormalized vacuum expectation value $\langle\phi_R\rangle$ is obtained from

$$\langle\phi_R\rangle = \frac{1}{\sqrt{Z}} \langle|\phi|\rangle. \quad (6.2.2)$$

In the broken phase, the renormalized coupling can then be defined as

$$\lambda_R = 3m_R^2 \langle\phi_R\rangle^2. \quad (6.2.3)$$

These quantities enter the definition of the renormalized topological charge discussed below.

6.3 Aperiodic Boundary Conditions (APBC)

We impose Aperiodic Boundary Conditions (APBC) in one direction,

$$\phi(x+L) = \phi(x), \quad (6.3.1)$$

while PBC are kept for the second direction. Hence, the field interpolates between two classical vacua across the lattice, therefore the configurations have a kink or antikink. As in [5], we define the following order parameter:

$$\phi_{\text{diff}} = \frac{1}{2} (\phi_{L-1} - \phi_0), \quad (6.3.2)$$

where ϕ_0 is the expectation value of the field at site 0, computed by averaging over all configurations generated with APBC (similary for ϕ_{L-1}). In the symmetric phase, ϕ_{diff} vanishes, while in the broken phase it takes a nonzero value, reflecting the presence of a kink.

We can define a bare topological charge on the lattice (see [5]) as follows:

$$Q_0 = \sqrt{\frac{\lambda_0}{3m_0^2}} \phi_{\text{diff}}, \quad (6.3.3)$$

where λ_0 and m_0 are the bare lattice coupling and bare mass. Q_0 is not quantized and deviates from ± 1 in the broken phase.

To get the quantization back, a renormalized topological charge is introduced ([5]):

$$Q_R = \sqrt{\frac{\lambda_R}{3m_R^2}} \phi_{R,\text{diff}}, \quad \phi_{R,\text{diff}} = \frac{1}{\sqrt{Z}} \phi_{\text{diff}}. \quad (6.3.4)$$

Using Equation 6.2.3, we obtain a simpler expression:

$$Q_R = \frac{\phi_{\text{diff}}}{\langle\phi\rangle}. \quad (6.3.5)$$

In the broken phase, Q_R approaches ± 1 within statistical errors, corresponding to kink and antikink sectors, while in the symmetric phase it remains close to zero.

Chapter 7

Results

This section presents the implementation of the theory into Python and Rust code and the results of various analyses.

7.1 Details about the Implementation

The code can be found under the following link:

<https://github.com/FiPf/LFT/tree/main/Final%20Project>

All of the code is in the subfolder "Final Project", please ignore all the other subfolders.

7.1.1 Description of the Python Code

The Python code is split into various subfolders, which serve as packages:

- **algorithms:** This package contains an abstract base class to run simulations with an unspecified algorithm. There are two classes (for Metropolis and HMC), which inherit from this abstract base class and are used to run Metropolis and HMC simulations. Finally, there is a file containing the two integrator choices for the HMC algorithm (leapfrog or omelyan2 integrator).
- **physics:** This file contains all functions regarding the physics of this project. The first file contains the action of the ϕ^4 -theory, its derivative, a function to calculate the hamiltonian of the system and an enum with the boundary condition choices (either periodic or aperiodic).
- **stats:** This package contains a file with all the statistical analysis functions used for this project. The file intended for logging is currently empty, but the code could be extended to contain loggers.
- The main file is a Jupyter Notebook, where all of the results are displayed. It accesses the other files through the packages described above.

7.1.2 Description of the Rust Code

As shown in Section ??, the HMC algorithm was extremely slow in Python, despite being mostly vectorized. It is possible that the object oriented structure of the code contributes to the slowness. Since rewriting the code in a faster way would be a considerable effort and only lead to a small improvement, I decided to instead translate the slowest parts into Rust.

Rust is much faster than Python, because Rust is a compiled ahead-of-time to machine code, while Python is an interpreted language (bytecode executed by the CPython virtual machine). This alone can lead to a speedup of a factor 10 – 100x. Rust also has other useful features that make coding typesafe and less chaotic than in Python. [13]

The following parts of the code were translated from Python to Rust:

- The HMC algorithms, together with the two integrators.

- The Metropolis algorithm.
- Some of the physics functions, for example the action and it's gradient, as they were used for HMC.

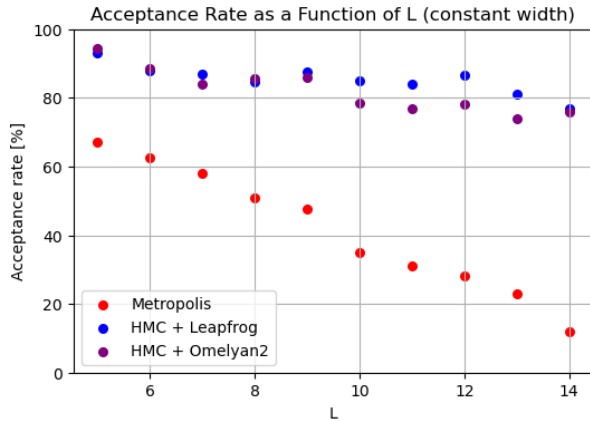
The structure was kept similar to the Python version. I used PyO3 to be able to import the Rust functions as a package and then use inside my Jupyter Notebook, together with Python code for analysis and plotting. [10]

Note that translating alone can lead to a great speedup. Since I am still learning Rust, this was a pure translation exercise. I did not use any special features of Rust or payed attention to writing efficient code.

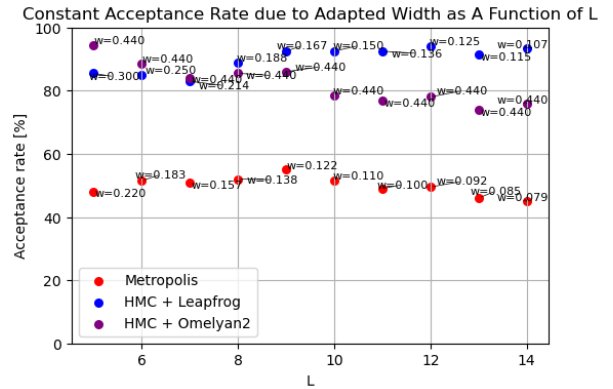
7.2 Tuning the Algorithms to the Correct Acceptance Rate

In the Metropolis algorithm, the proposal step size is tuned to achieve an acceptance rate of approximately 50%. This value represents a compromise between making sufficiently large moves in configuration space and maintaining a reasonable probability of acceptance. If the acceptance rate is too high, the proposed updates are very small and the Markov chain explores the configuration space inefficiently. If it is too low, most proposals are rejected and the simulation again becomes inefficient.

In contrast, the HMC algorithm is based on approximate Hamiltonian dynamics, where rejections arise solely from numerical integration errors. Since each HMC proposal involves a full molecular dynamics trajectory, rejected updates are computationally expensive. For this reason, HMC is optimally operated at much higher acceptance rates, typically around 90%. Higher acceptance ensures accurate energy conservation and efficient exploration of phase space, especially when using higher-order integrators such as the omelyan2 scheme.



(a) The dependence of the acceptance rate on the lattice size L . The following widths were used: Metropolis $w = 0.14$ / HMC + Leapfrog $w = 0.2$ / HMC + Omelyan2 $w = 0.44$



(b) The widths were adapted as a function of L to achieve an acceptance rate of 50% for Metropolis and 90% for HMC. The following width functions were used: Metropolis $w = 1.1/L$ / HMC + Leapfrog $w = 1.5/L$ / HMC + Omelyan2 $w = 0.44$ (const).

Figure 7.2.1: Tuning the Acceptance Rates for Different Algorithms

The same tuning parameters were used with the Rust implementation. This was a simple test to see whether the Rust version produces the same results as the Python version consistently.

7.3 Monte Carlo Chain Visualization

In Figure 7.3.1, the Monte Carlo chain of the simulation with different algorithms is shown. The observable displayed is the mean field. This plots are not very useful.

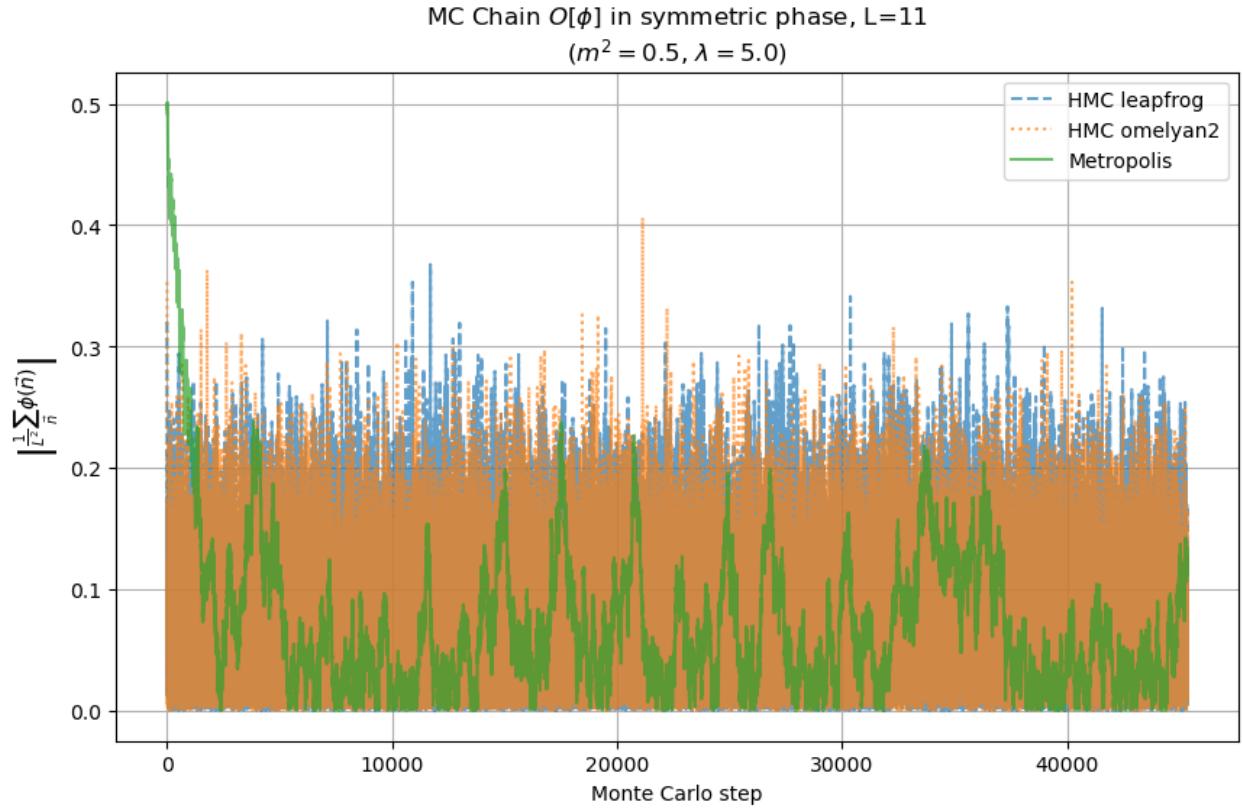


Figure 7.3.1: Monte Carlo Chain of the Mean Field, Simulation using Different Algorithms

The Metropolis algorithm has the longest thermalization phase.

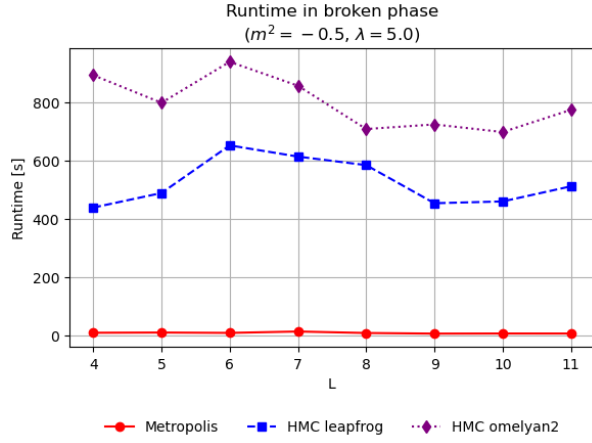
7.4 Computational Performance of the Algorithms

In this section, the computational performance of the different simulation algorithms is compared. The runtimes are shown as a function of lattice size. In addition, the impact of the programming language is investigated comparing Python and Rust versions of the same algorithm. Since lattice simulations are limited by the computational cost, choosing the right algorithm and programming language is crucial.

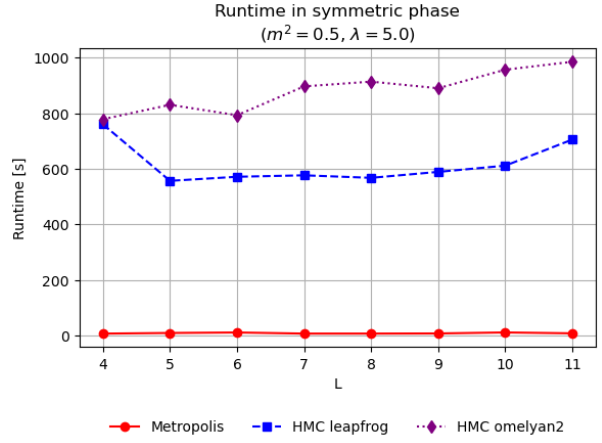
The Metropolis and HMC algorithms differ also in their structure and where most of the computational cost is used. While Metropolis updates are local and inexpensive per step, they suffer from long autocorrelation times. HMC, on the other hand, performs global updates using molecular dynamics trajectories, which are computationally more expensive per iteration but typically yield much faster decorrelation. The statistical independence between configurations is investigated in Section 7.5.

Furthermore, Python and Rust differ fundamentally in other aspects. Python offers ease of implementation and flexibility but incurs significant overhead due to interpretation and dynamic typing. In contrast, Rust compiles to optimized native code and allows exact control over memory and numerical operations, making it well suited and fast for computationally intensive simulations. In this final project, the computationally intensive algorithms were implemented in Rust and then imported to Python using PyO3. This allows to still use the great Python libraries such as numpy and matplotlib for the analysis and plotting.

The following Figure 7.4.1 shows the runtimes for the different algorithms in the broken and in the symmetric phase of ϕ^4 -theory using the algorithms implemented in Python.



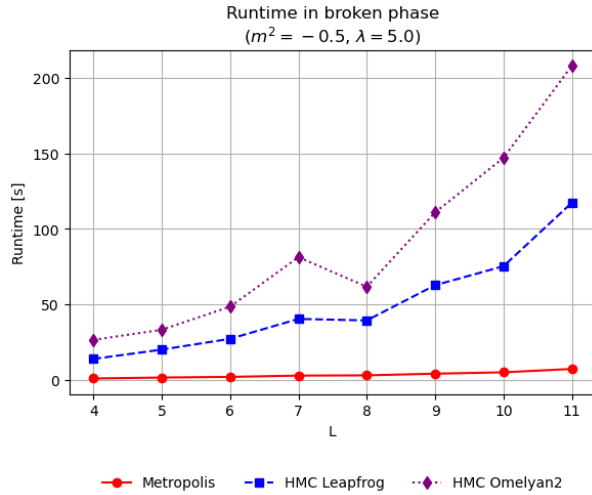
(a) Broken phase with negative mass



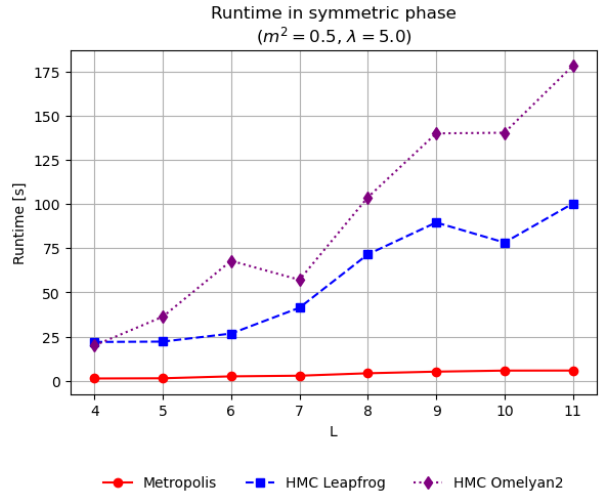
(b) Symmetric phase with positive mass

Figure 7.4.1: Python runtimes

The following Figure 7.4.2 shows the runtimes for the different algorithms in the broken and in the symmetric phase of ϕ^4 -theory using the algorithms implemented in Rust.



(a) Broken phase with negative mass



(b) Symmetric phase with positive mass

Figure 7.4.2: Rust runtimes

Comments on the results

- Very clearly visible is the speedup achieved through translating the Python code into Rust, the Rust runtimes are a factor of 6–8x faster. This was achieved by translating to a more suitable programming language, no efficiency improvements in the algorithms were implemented.
- The Metropolis algorithm is the fastest overall, in both Python and Rust, In the Python implementation, the runtime of Metropolis is nearly constant, while in Rust, we observe a slight increase with increasing lattice size L .
- The HMC algorithm is significantly slower than the Metropolis algorithm, where the leapfrog integrator is faster than the omelyan2 integrator used in HMC.
- In the Python version, the runtime of HMC for both integrators are more or less constant with the lattice size L for unclear reasons. For the Rust versions of the HMC algorithms, we observe a linear

increase of runtime with L . This can be explained by the fact that HMC has a larger configuration to update and the equations of motion are more expensive to solve.

- In addition, it can be observed that the runtime in the symmetric phase is higher than in the broken phase of the ϕ^4 -theory. This can be understood in terms of critical slowing down and the structure of field configurations in the two regimes.

In the symmetric phase, the system is close to the critical point where the correlation length becomes large. Field updates become more strongly correlated over large distances, the state space is explored more slowly. In contrast, in the broken phase the field fluctuates around one of the two degenerate minima of the potential. The presence of a non-zero vacuum expectation value effectively introduces a mass scale that suppresses long-range correlations. This leads to faster decorrelation of configurations and improved sampling efficiency, reducing the computational cost per independent configuration. [8]

7.5 Measuring Errors and Autocorrelations

7.5.1 Errors

Accurate error estimations are essential to quantify the reliability of computed observables and to compare benefits of different algorithms. This subsection describes how the statistical errors in Metropolis and HMC using both the leapfrog and omelyan2 integrator evaluated as a function of lattice size L .

The following Figure 7.5.1 shows the statistical errors of both phases resulting from the Python implemented algorithms.

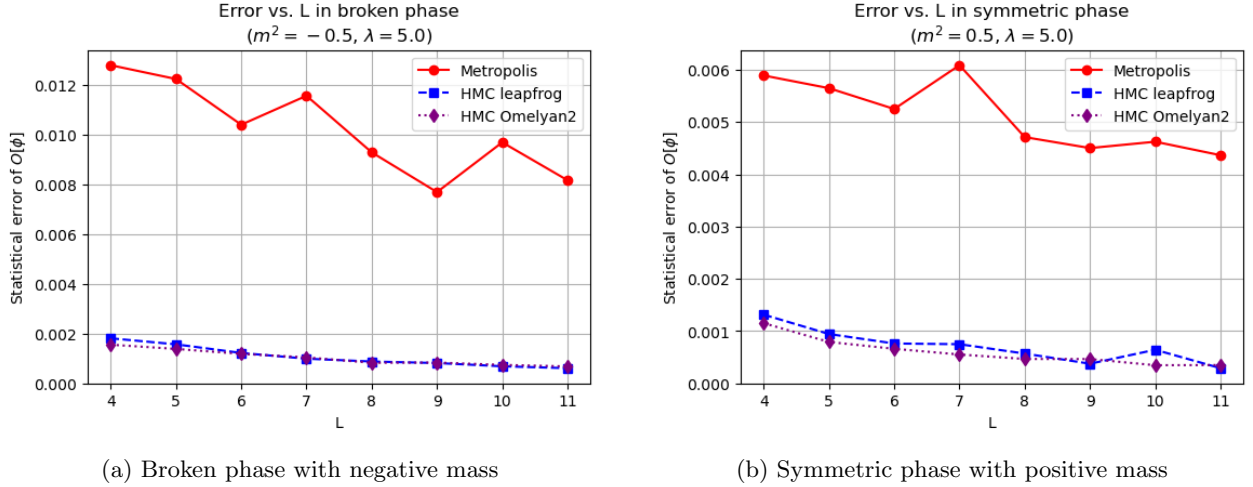
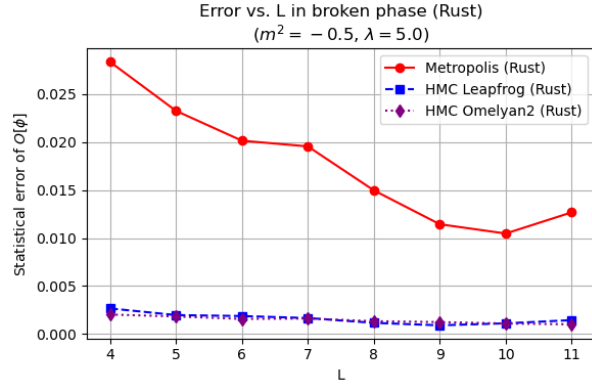
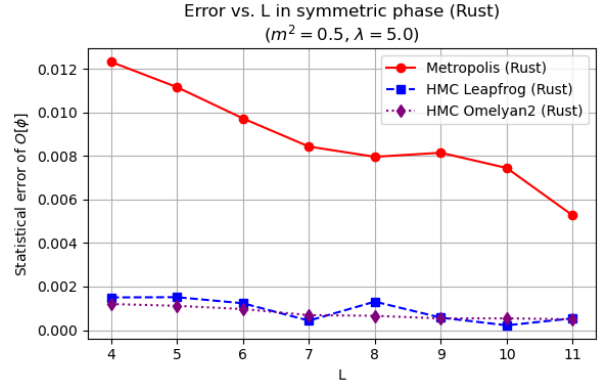


Figure 7.5.1: Statistical Errors as a function of lattice size L for the broken and symmetric phase, algorithms in Python

The following Figure 7.5.2 shows the statistical errors of both phases resulting from the Rust implemented algorithms.



(a) Broken phase with negative mass



(b) Symmetric phase with positive mass

Figure 7.5.2: Statistical Errors as a function of lattice size L for the broken and symmetric phase, algorithms in Rust

7.5.2 Autocorrelations

Autocorrelations measure the memory of a Markov chain and quantify the dependence of successive configurations. High autocorrelation reduces the effective number of independent samples, increasing statistical errors. We discuss the computation of integrated autocorrelation times and their impact on the precision of measured observables.

7.6 Results with the Topological Charge

Bibliography

- [1] Dr. Tim Brereton. *Methods of Monte Carlo Simulation*. <https://www.math.ucdavis.edu/~tracy/courses/math135A/UsefullCourseMaterial/lawLargeNo.pdf>. Accessed: 2025-07-11.
- [2] Wikipedia contributors. *Autocorrelation*. <https://en.wikipedia.org/wiki/Autocorrelation>. Accessed December 2025. 2025.
- [3] Wikipedia contributors. *Markov chain Monte Carlo*. Accessed September 2025. 2025. URL: https://en.wikipedia.org/wiki/Markov_chain_Monte_Carlo.
- [4] Wikipedia contributors. *Standard error*. Accessed September 2025. 2025. URL: https://en.wikipedia.org/wiki/Standard_error.
- [5] Asit K. De et al. “Topological charge in 1 + 1 dimensional lattice phi4 theory”. In: *Physical Review D* 72.9 (Nov. 2005). ISSN: 1550-2368. DOI: [10.1103/PhysRevD.72.094504](https://doi.org/10.1103/PhysRevD.72.094504). URL: <http://dx.doi.org/10.1103/PhysRevD.72.094504>.
- [6] Simon Duane et al. “Hybrid Monte Carlo”. In: *Physics Letters B* 195.2 (1987), pp. 216–222. DOI: [10.1016/0370-2693\(87\)91197-X](https://doi.org/10.1016/0370-2693(87)91197-X).
- [7] Paolo Favaro. *Machine Learning*. Bachelors Course in Computer Science. 2023.
- [8] Christof Gattringer and Christian B. Lang. *Quantum Chromodynamics on the Lattice: An Introductory Presentation*. Lecture Notes in Physics, Vol. 788. Springer Berlin / Heidelberg, 2010. ISBN: 978-3-642-01849-7.
- [9] I. P. Omelyan, I. Mryglod, and R. Folk. “On the construction of high-order force-gradient algorithms for integration of motion in classical and quantum systems”. In: *Computer Physics Communications* 151.3 (2003), pp. 272–314. DOI: [10.1016/S0010-4655\(02\)00754-3](https://doi.org/10.1016/S0010-4655(02)00754-3).
- [10] PyO3 Project Contributors. *PyO3: Rust bindings for the Python interpreter*. Accessed: 2025-03-08. 2025. URL: <https://pyo3.rs/>.
- [11] Matthew D. Schwartz. *Quantum Field Theory and the Standard Model*. Cambridge University Press, 2014. DOI: [10.1017/CB09780511813456](https://doi.org/10.1017/CB09780511813456).
- [12] Tetsuya Takaishi and Philippe de Forcrand. “Testing and tuning symplectic integrators for Hybrid Monte Carlo algorithm in lattice QCD”. In: *Phys. Rev. E* 73 (2006), p. 036706. DOI: [10.1103/PhysRevE.73.036706](https://doi.org/10.1103/PhysRevE.73.036706). arXiv: [hep-lat/0505020](https://arxiv.org/abs/hep-lat/0505020) [hep-lat].
- [13] The Rust Project Developers. *Rust Cookbook*. Rust Community, 2024. URL: <https://rust-lang-nursery.github.io/rust-cookbook/>.
- [14] Uwe-Jens Wiese. *Statistical Thermodynamics*. <https://wiese.itp.unibe.ch/lectures/thermodyn.pdf>. Lecture Notes, Institute for Theoretical Physics, University of Bern. 2010.
- [15] Wikipedia Contributors. *Five-point stencil* — *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Five-point_stencil. Accessed: 2025-12-07. 2025.